LESSON 11 · CHEAT SHEET

Grouping & Partitioning

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. `groupingBy(classifier)` = `Map<K, List<T>>` in a **HashMap**, so the order is not defined. 8
2. 3-argument form: `groupingBy(k, TreeMap::new / EnumMap, downstream)`.
3. **Downstream** = what to do with each bucket: counting, mapping, reducing, maxBy, joining…
4. `filter` before grouping **removes keys**; `filtering` keeps them with 0. 9
5. `partitioningBy` always has **both** keys, false first. 8
6. maxBy gives an Optional; unwrap with `collectingAndThen(…, o -> o.orElseThrow())`.
7. Several dimensions? Nested maps for drill-down, a **record key** for flat reports.

Canonical snippets

1 · Count per key, sorted

```
customers.stream().collect(groupingBy(
    Customer::city, TreeMap::new, counting()));
// {Bangkok=1, Colombo=1, Jakarta=2,
//  Kuala Lumpur=2, Singapore=4}
```

2 · Keep empty groups

```
groupingBy(Payment::method, TreeMap::new,
    filtering(p -> p.status() == FAILED, counting()))
// {CARD=0, PAYNOW=1, PAYPAL=1}
```

3 · Largest per key, unwrapped

```
groupingBy(Transaction::accountNumber,
    TreeMap::new, collectingAndThen(
        maxBy(comparing(Transaction::amount)),
        o -> o.orElseThrow().id()))
// {ACC-1001=T018, ACC-1002=T005, …}
```

Downstream menu

Downstream	Each bucket becomes	Since
`(none) / toList()`	List<T>	8
`counting()`	Long	8
`summingInt / averagingInt`	Integer / Double	8
`reducing(ZERO, amount, add)`	BigDecimal total	8
`mapping(f, toCollection(TreeSet::new))`	sorted set of f	8
`mapping(name, joining(", "))`	String	8
`maxBy / minBy`	Optional<T>	8
`collectingAndThen(c, f)`	f(result)	8
`groupingBy(k2, …)`	nested Map	8
`filtering(p, c)`	c over matching items, 0/empty kept	9
`flatMapping(f, c)`	c over flattened items	9
`teeing(c1, c2, merger)`	two answers, one pass	12

partitioningBy vs groupingBy (real output)

```
flights where booked > seats (none are):
partitioningBy = {false=8, true=0}
groupingBy     = {false=8}       // true key missing
```

Mini diagram: classify → buckets → downstream



Real results from the course code

Call	Result
seats per origin	{CGK=162, SIN=1916}
partition SUCCESS totals	{false=1430.65, true=2999.19}
teeing per method 12	{CARD=5/2070.29, PAYNOW=4/1824.30, PAYPAL=3/535.25}
top 3 categories by count	[TRANSFER=15, DINING=10, SALARY=8]

Worked example: debit report	Result
first line	TRANSFER: 8 debits, total 7190.92, largest T042 1432.93
last line	DINING: 10 debits, total 839.28, largest T020 170.53
Exercise: avg premium	LIFE=2125.00 · HEALTH=1032.88 · MOTOR=1275.25 · TRAVEL=197.50 · HOME=782.63

Anti-patterns → fixes

Don't	Do
filter, then group, when every key must show	`filtering(p, counting())` 9
`maxBy`, then `.get()` later	`collectingAndThen(maxBy, unwrap)`
print the default HashMap	`TreeMap::new` / `EnumMap`

Don't	Do
`summingDouble(amount)`	`reducing(ZERO, amount, add)`
3-level nested maps	a record key, e.g. `CityTier(city, tier)`
a loop with `map.get(k) == null`	`groupingBy` / `computeIfAbsent`

Lesson 11 · sticky notes

Cut out, stick on your wall, and read one every morning.

groupingBy = buckets

classifier picks the bucket
downstream fills it

Default = HashMap

No order! Use
TreeMap::new / EnumMap

filter vs filtering [9]

filter first → key gone
filtering → key = 0

partitioningBy

ALWAYS false AND true
{false=8, true=0}

maxBy is Optional

collectingAndThen(
maxBy(..), o -> o.orElseThrow())

Money per group

`reducing(ZERO,`
`amount, BigDecimal::add)`

flatMapping [9]

customer → many accounts
inside the bucket

Many dimensions?

nested map = drill-down
record key = flat report